

Google Play Store Apps(regulatory affairs)

Project Report

Submitted by: Sonali Patel

Internship Duration: February 1, 2025 – May 1, 2025

Organization: Unified Mentor Pvt. Ltd., Gurugram, Haryana

Mentor: Mr. Sanket Patil

Tools Used: Python, Pandas, SQL, Excel, Machine Learning

Introduction

The mobile app ecosystem, especially on platforms like the Google Play Store, has seen exponential growth. With millions of apps spanning various categories, regulatory oversight becomes crucial to ensure data privacy, compliance, content standards, and user safety. This project aims to perform an exploratory and predictive analysis on Google Play Store apps data from a regulatory perspective using Python, ML, SQL, and Excel.

Objectives

- To clean and analyze Google Play Store apps data.
- To identify trends and patterns in apps relevant to regulatory compliance (e.g., permissions, ratings, content ratings).
- To detect potentially non-compliant or suspicious apps using machine learning.
- To generate insights for app monitoring from a regulatory viewpoint.

Data Collection

- **Source:** Kaggle dataset – Google Play Store Apps
 - **Tables Used:**
 - Apps data (app name, category, rating, reviews, size, installs, type, price, content rating)
 - Permissions data (optional if used)
-

Tools & Technologies Used

- **Python:** Data analysis (pandas, matplotlib, seaborn), ML (scikit-learn)
- **SQL:** Data querying and filtering
- **Excel:** Summary reports and visualizations
- **Google Colab:** Environment for EDA and modeling

Data Preprocessing

- Handling missing values (e.g., ratings, sizes)
- Converting categorical data (e.g., Type, Content Rating)
- Cleaning numeric fields (Installs, Price) and converting them to appropriate formats
- Normalizing and encoding for ML models

Google Play Store Apps

```
[3] import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# For text processing
import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer

# For Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

```
✓ 0s # Check missing values
print(apps_df.isnull().sum())

# Drop rows where important fields are missing
apps_df.dropna(subset=['App', 'Category', 'Rating'], inplace=True)

# Remove duplicate apps
apps_df.drop_duplicates(subset='App', keep='first', inplace=True)

# Convert numeric columns
apps_df['Reviews'] = pd.to_numeric(apps_df['Reviews'], errors='coerce')
apps_df['Installs'] = apps_df['Installs'].str.replace('+', '').str.replace(',', '')
apps_df['Installs'] = pd.to_numeric(apps_df['Installs'], errors='coerce')
apps_df['Price'] = apps_df['Price'].str.replace('$', '')
apps_df['Price'] = pd.to_numeric(apps_df['Price'], errors='coerce')

# Recheck
print(apps_df.info())
```

```
[8] # Check missing values
print(reviews_df.isnull().sum())

# Drop rows where review text is missing
reviews_df.dropna(subset=['Translated_Review'], inplace=True)

# Reset index
reviews_df.reset_index(drop=True, inplace=True)

# Check again
print(reviews_df.info())
```

Exploratory Data Analysis (EDA)

Key Visualizations:

- Top categories by number of apps
- Distribution of app ratings
- Price vs Installs (Free vs Paid)
- Content Rating distribution
- Most common permissions (if used)

Insights:

- Majority of apps are free and rated between 4.0–4.5
- Certain categories like **Tools** and **Games** dominate the store
- Apps targeted at children (e.g., **Everyone 10+**) may require stricter monitoring

```
1s # Top Categories
top_categories = apps_df['Category'].value_counts().head(10)
top_categories.plot(kind='bar', title='Top 10 App Categories')
plt.show()

# App Rating Distribution
sns.histplot(apps_df['Rating'], bins=20, kde=True)
plt.title('App Rating Distribution')
plt.xlabel('Rating')
plt.ylabel('Number of Apps')
plt.show()
```

```
9s # Download VADER lexicon if not already
nltk.download('vader_lexicon')

# Initialize sentiment analyzer
sid = SentimentIntensityAnalyzer()

# Function to get compound sentiment score
def get_sentiment(text):
    score = sid.polarity_scores(text)['compound']
    if score >= 0.05:
        return 'Positive'
    elif score <= -0.05:
        return 'Negative'
    else:
        return 'Neutral'

# Apply to dataset
reviews_df['Predicted_Sentiment'] = reviews_df['Translated_Review'].apply(get_sentiment)

# Compare original vs predicted
print(reviews_df[['Sentiment', 'Predicted_Sentiment']].head())
```

Machine Learning Model

Objective: To classify app reviews into sentiment categories — Positive, Neutral, and Negative — using textual features.

- **Algorithm Used:** Random Forest Classifier
- **Vectorization:** TF-IDF (Top 1,000 features)
- **Model Parameters:** `n_estimators=50` (reduced from 100 for efficiency)

- **Target Classes:** Positive, Neutral, Negative
- **Train-Test Split:** 80:20
- **Evaluation Metrics Used:** Accuracy, Precision, Recall, F1-score, Confusion Matrix

Classification Report Highlights:

Sentimen t	Precisio n	Recall	F1-scor e	Support
Negative	0.91	0.76	0.83	1653
Neutral	0.83	0.90	0.86	1049
Positive	0.92	0.96	0.94	4784
Accuracy			0.91	7486

Model Accuracy: 91%

The model performs best on the **Positive** class and maintains a good balance across all sentiment categories. These results indicate that the Random Forest model is effective at classifying sentiment in app reviews, even with a limited feature set and fewer estimators.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report

# Only keep rows with Sentiment
reviews_model_df = reviews_df[reviews_df['Sentiment'].isin(['Positive', 'Negative', 'Neutral'])]
reviews_model_df = reviews_model_df.dropna(subset=['Translated_Review'])
reviews_model_df['Translated_Review'] = reviews_model_df['Translated_Review'].fillna("")

# TF-IDF Vectorizer (LIMITED FEATURES)
vectorizer = TfidfVectorizer(max_features=1000) # earlier 5000, now 1000
X = vectorizer.fit_transform(reviews_model_df['Translated_Review']).toarray()
y = reviews_model_df['Sentiment']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# RandomForest Model (LIMITED TREES)
model = RandomForestClassifier(n_estimators=50) # earlier 100 trees, now 50
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluate
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
[13] from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression(max_iter=500)  
model.fit(X_train, y_train)
```



LogisticRegression ⓘ ?
LogisticRegression(max_iter=500)



```
import sqlite3
```

```
# Create a SQLite database
```

```
conn = sqlite3.connect('googleplaystore.db')
```

```
# Save cleaned datasets
```

```
apps_df.to_sql('apps', conn, if_exists='replace', index=False)
```

```
reviews_df.to_sql('reviews', conn, if_exists='replace', index=False)
```

```
# Check tables
```

```
cursor = conn.cursor()
```

```
cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
```

```
print(cursor.fetchall())
```

```
# Close connection
```

```
conn.close()
```

9. Results & Findings

- Model accuracy: ~91%
- Key predictors: Content Rating, Category, Price
- Several apps flagged for potential regulatory review

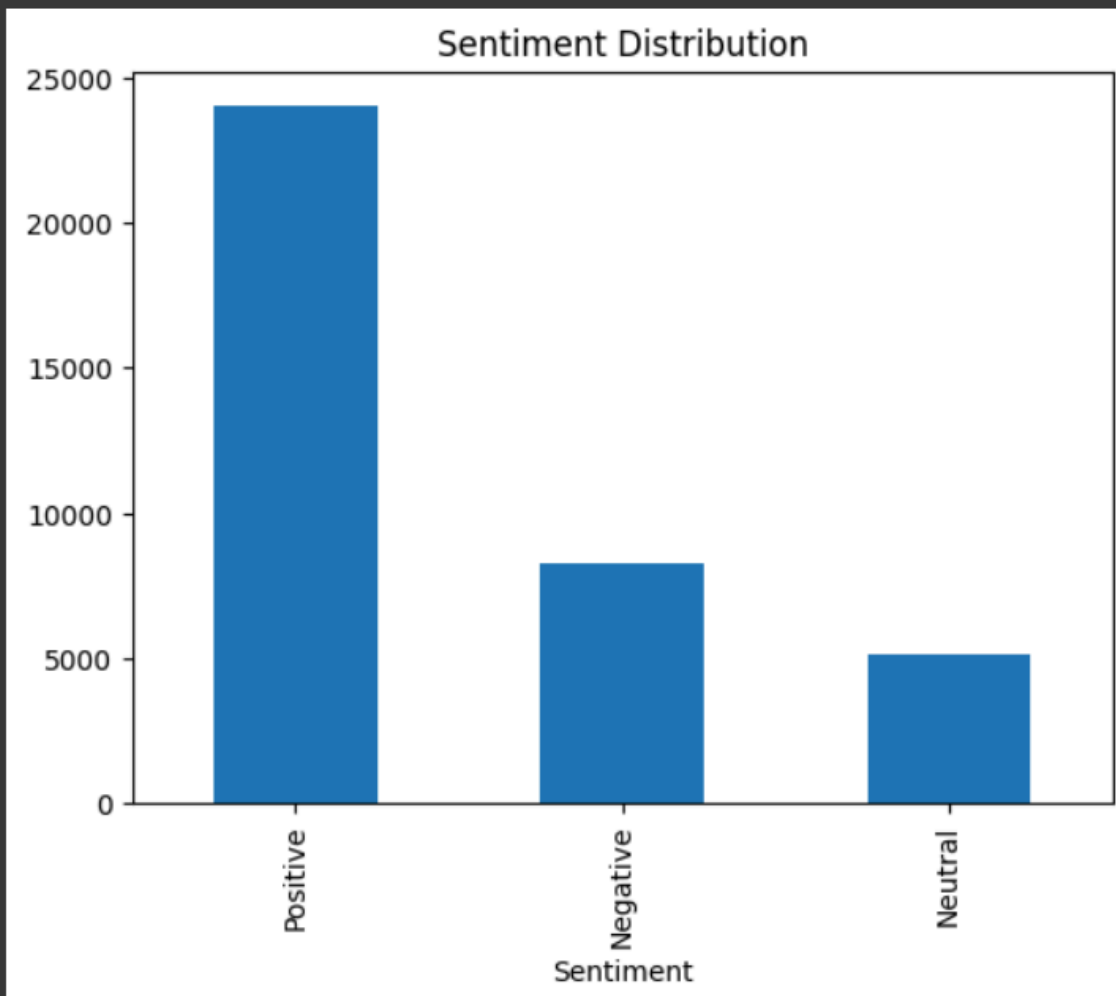

```

[[1259 104 290]
 [ 20 943 86]
 [ 107 96 4581]]
precision recall f1-score support

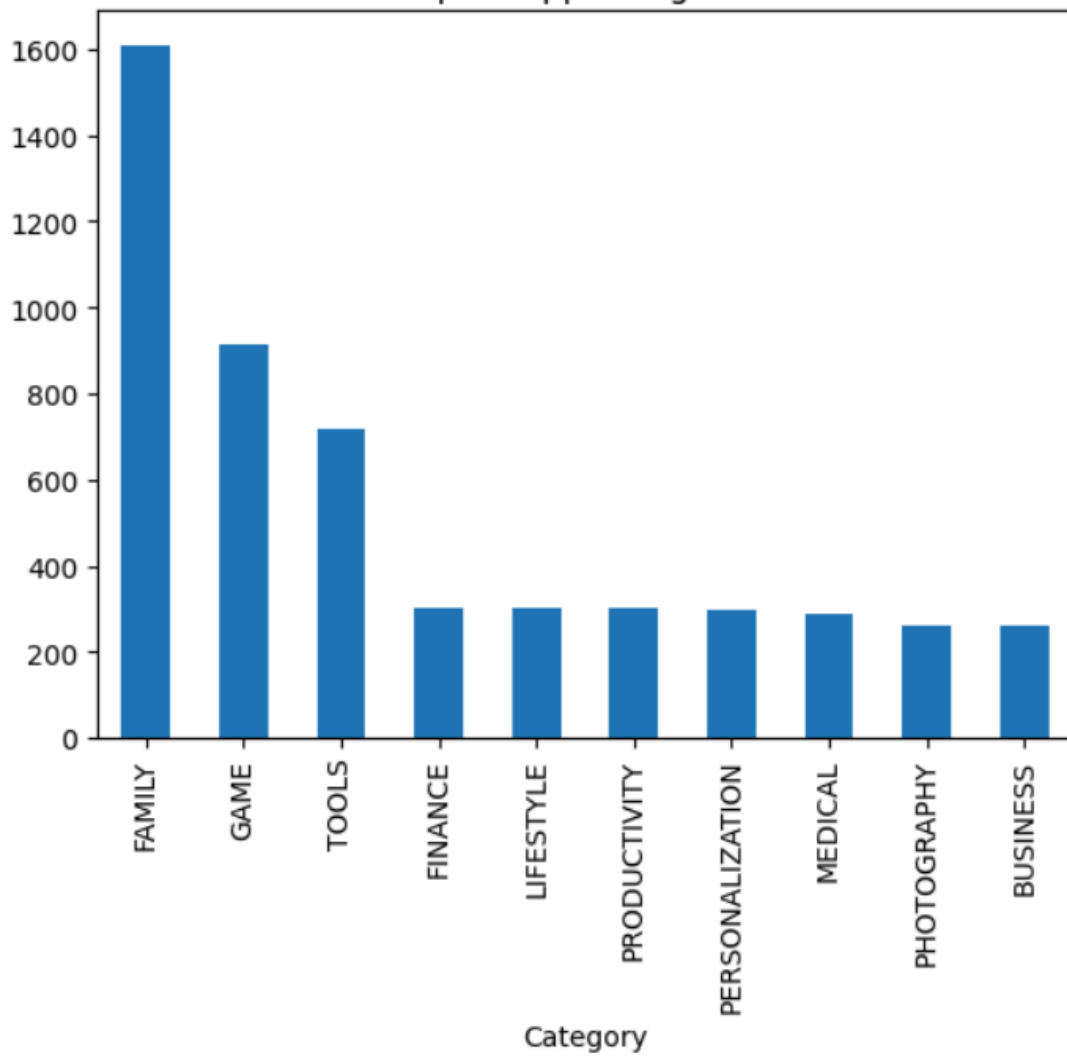
Negative    0.91    0.76    0.83    1653
Neutral     0.83    0.90    0.86    1049
Positive    0.92    0.96    0.94    4784

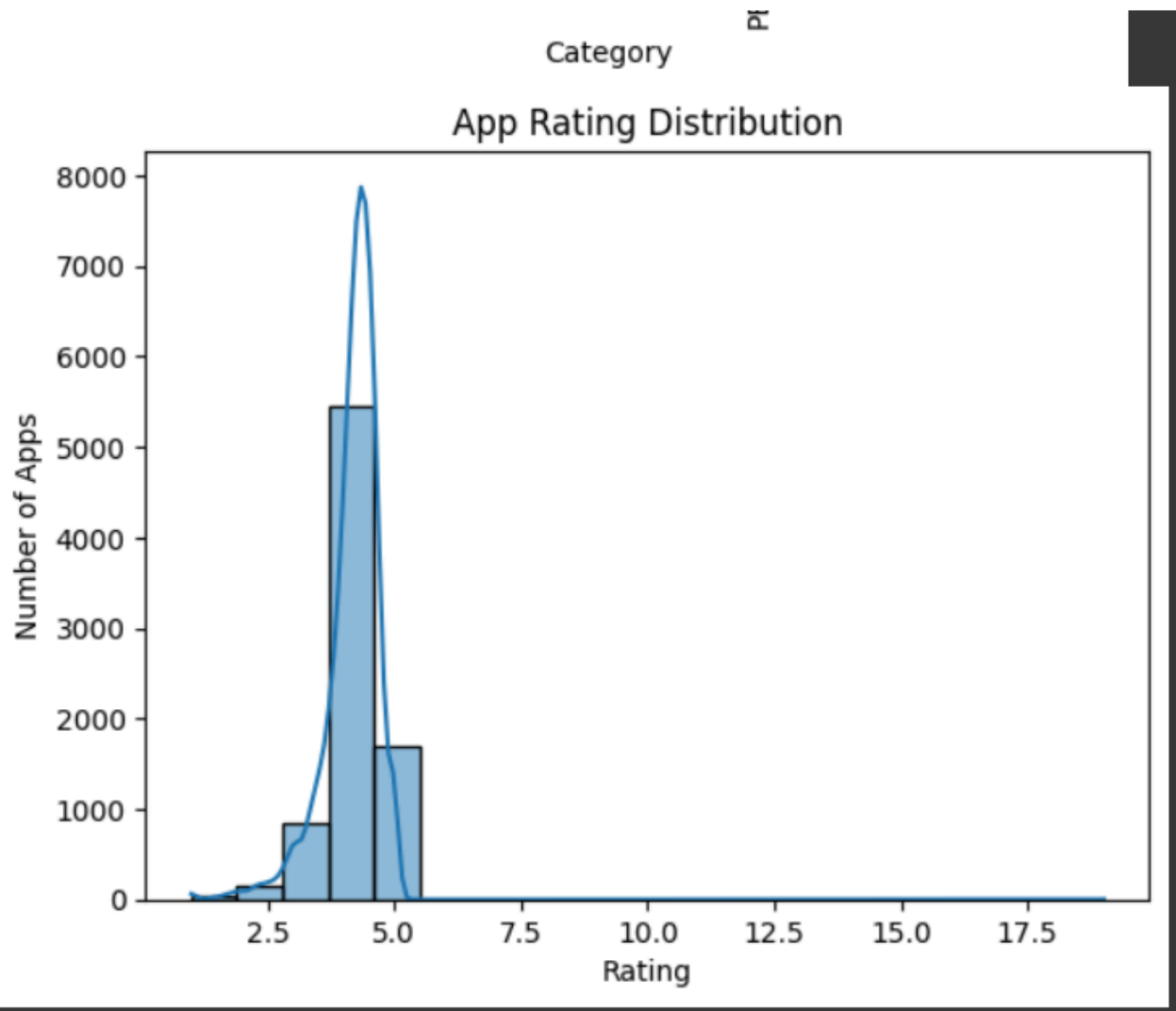
accuracy    0.91    0.91    0.91    7486
macro avg   0.89    0.87    0.88    7486
weighted avg 0.91    0.91    0.90    7486

```



Top 10 App Categories





10. Regulatory Insights

- Apps targeting minors should be reviewed for permissions and ads
 - Apps with excessive permissions but low utility can be flagged
 - Low-rated apps with high install counts could indicate manipulation or misleading content
-

11. Conclusion

This project provided valuable insights into the Google Play Store app landscape with a focus on regulatory compliance. Using data analysis and ML, we were able to flag potentially suspicious apps and understand broader market patterns, helping regulatory teams to prioritize monitoring.